

SIH 2026 – PS26124 Extension Roadmap

"AI-Powered Mobile Urban Intelligence Platform Using Public Transport Fleet" (Bharat Electronics Limited)

This maps every requirement in the problem statement to a **real, publicly available dataset**, shows exactly how your existing IMU/IRI/pothole system fits in, and is honest about what's realistically buildable vs. what should be framed as a roadmap item. No feature here is invented — each one is backed by a dataset or a known technique cited below.

1. The core reframe

PS26124 is fundamentally a **multi-camera, edge-AI-on-buses + centralized GIS analytics** problem. Skimming the "Description" and "Expected Solution" sections, the graded capabilities are:

1. Road defect detection (potholes, damaged roads, missing dividers, missing zebra crossings, damaged/missing signboards, waterlogging)
2. Vehicle density estimation, classification, counting
3. Traffic bottleneck identification
4. Vulnerable pedestrian detection (school children crossing)
5. Hit-and-run / rash-driving detection → vehicle tracking → ANPR with **confidence score**, timestamp, GPS → secure alert
6. Centralized platform: GIS map, congestion heatmaps, infra-deficiency map, O-D pattern analysis, route-delay estimation

Your IMU + IRI + 2-level pothole classifier is a real, working asset — but on its own it only touches capability #1 (and only the "pothole/road roughness" slice of it). To be competitive you need the camera + fleet-analytics layers explicitly asked for. The goal below isn't to bolt on random buzzword features — it's to (a) find the two or three places your existing IMU work is a genuine, non-obvious advantage, and (b) fill the rest with modules that have real training data behind them.

2. Your IMU pipeline's two legitimate re-uses (your actual differentiator)

Most teams tackling this PS will start from zero on every module. You won't, if you position the IMU system correctly:

a) Bandwidth-saving trigger / sensor-fusion validator. The Expected Solution explicitly asks for "minimizing bandwidth through intelligent edge processing." Running 4 camera streams through vision models continuously is expensive on edge hardware and bandwidth-heavy to sync centrally. Your IMU signal is cheap to process (it's just accelerometer/gyro math) and already tells you *when* the road surface is anomalous. Use it as a gate: only trigger the full vision pothole/crack detector (and only upload the flagged frame + metadata, not raw video) when the IMU crosses a roughness threshold. This is a concrete, defensible answer to a requirement most teams will only pay lip service to.

b) Harsh-event detector for hit-and-run / rash driving. Commercial fleet dashcams (Samsara, Motive) already use a G-sensor/accelerometer spike to trigger "Collision Evidence Capture" — i.e., the exact moment of impact or hard braking is detected by the IMU, not the camera. You already have this math built for pothole/IRI scoring; the same jerk/deceleration-spike detection, repurposed with different thresholds, tells you *when* to run ANPR on the surrounding vehicles instead of running plate recognition continuously on every vehicle in frame. This directly reduces compute load and false positives.

c) The strongest single differentiator: auto-labeling your camera dataset from your IMU ground truth. You already have GPS + timestamp + severity-labeled pothole events from real-world IMU data. If you sync a forward camera to the same device, you can auto-generate weak labels for the corresponding video frames at each event's GPS/timestamp — bootstrapping a camera-based pothole/road-damage dataset without manual annotation. This is a legitimate weak-supervision technique, not a gimmick, and it's a claim almost no other team in the room will be able to make truthfully. Lead your pitch with this.

3. Feature-by-feature build plan, with real datasets

PS requirement	What to build	Real dataset(s) you can actually train on
Potholes, cracks, damaged	Vision detector (YOLO-family) fused with your IMU signal	RDD2020 / RDD2022 (CRDDC challenge): 26k–47k dashcam images from India, Japan, Czech Republic, Norway, US, China; pothole class D40 explicitly included, with a real India-collected

roads		subset (originally curated with IIT Roorkee). Free, YOLO-annotatable.
Missing zebra crossing / missing lane markers	Detection of faded/absent markings	The same RDD2020/2022 dataset already has "crosswalk blur" (D43) and "white line blur" (D44) classes — worn/faded-marking detection is the realistic, buildable version of "missing" markings (no public dataset of "never painted" markings exists, understandably). Be precise about this in your pitch — it's a strength, not a gap, if you name it correctly.
Damaged / missing traffic signboards	Detection + damage classification	IRTSD-Datasetv1 (5,141 images, 37 sign classes, 90 Indian cities, IEEE DataPort); DataCluster Labs Indian Traffic/Signboard datasets (2,000+ crowdsourced images, COCO/YOLO/VOC formats); INTSD (Indian night-time sign dataset) for low-light robustness. For the "missing" side specifically: MTSVD (Missing Traffic Signs Video Dataset) — 200 scenes purpose-built for missing-sign detection with contextual cues.
Vehicle density, classification, counting, bottleneck ID	Detection + multi-object tracking (ByteTrack/DeepSORT)	IDD / IDD-117K (India Driving Dataset, IIIT-H): ~95k+ images, 13 classes including autorickshaw, bus, truck, bicycle, rider — built specifically for unstructured Indian traffic. DriveIndia (TiHAN-IIT Hyderabad): ~67k images, 24 traffic-relevant classes, published YOLO baseline already hits 78.7% mAP50, so feasibility is proven, not assumed.
Vulnerable pedestrians / school-zone crossing	Pedestrian detection + geofencing, NOT age classification	Use IDD's "person"/"rider" classes for generic pedestrian detection, combined with school location POIs pulled from OpenStreetMap's Overpass API (free, real, downloadable) + time-of-day (school opening/closing hours) to flag "vulnerable zone" alerts. Deliberately avoid trying to classify a pedestrian's age from video — it's technically unreliable, there's no ethical public dataset for it, and geofencing + generic pedestrian density is both more buildable and the better engineering answer.
		"Indian Licence Plate Dataset in the wild"

Hit-and-run / rash driving + ANPR with confidence score	Tracking + trajectory-jerk detection (triggered by your IMU harsh-event signal) → two-stage ANPR (plate detect + OCR)	(16,192 images, 21,683 annotated plates); Roboflow "Indian Number Plate" datasets; Kaggle Indian vehicle/plate datasets. A published fine-tuned Indian-plate OCR model already reports 98%+ accuracy, so this is a realistic target, not a stretch. Attach the OCR model's native softmax/CTC confidence score directly — the PS literally asks for this, and most teams will forget to explicitly surface it.
Waterlogging	Segmentation / heuristic	Honest flag: this is the weakest-data feature. Public datasets (Puddle-1000 ~985 images, "Roadway Flooding Image Dataset" ~441 images) are small and mostly non-Indian. Recommend: a simple reflection/specular-highlight heuristic as a fallback, plus a small custom dataset collected during your own pilot runs in the monsoon season. Present this explicitly as a Phase-2/stretch item in your pitch rather than over-promising a robust detector — judges respect honesty about data gaps more than they punish it.
Centralized GIS platform: heatmaps, infra-deficiency map, O-D analysis, route delay	PostGIS + Leaflet/deck.gl dashboard, ingest existing GPS feed	Nearly all Indian public-service buses already carry AIS-140-compliant GPS/VLTD hardware (mandatory under MoRTH's Rule 125H since Jan 2019, integrated with the government's Vahan portal). Plug into this existing feed for location/route-delay/O-D analysis instead of pitching new GPS hardware — this is a strong feasibility argument because the backbone infrastructure is already deployed at scale.

4. Architecture sketch (put a version of this in your PPT)

BUS (onboard, edge box — Jetson-class SoC)

- ├ Cameras (front / side / rear / cabin) → per-position lightweight detector
| (YOLO fine-tuned per dataset above)
- ├ IMU (your existing pipeline) → (a) roughness/pothole trigger
| (b) harsh-event / collision trigger
| (c) auto-label generator (training time only)
- ├ Existing AIS-140 GPS/VLTD feed → location + timestamp tagging

- └ Event fusion layer: only compressed metadata + a still frame (not raw video) is uplinked, unless a human review is triggered

|



CENTRAL PLATFORM

- └ PostGIS + Leaflet/deck.gl: road-condition map, congestion heatmap
- └ Confidence-scored incident queue (ANPR reads, defect flags) for human review
- └ O-D / route-delay analytics computed from the AIS-140 feed
- └ Dashboards for transport authority / BEL command center

The "only upload compressed metadata + a still frame unless review is triggered" line is your direct, explicit answer to "minimizing bandwidth through intelligent edge processing" — say it in exactly those words in the PPT, since that's a phrase straight from the Expected Solution section.

5. Precedent to know — and how to differentiate from it

iRASTE (Intel, INAI, IIT-Hyderabad, CSIR-CRRI, Mahindra & Mahindra, Nagpur Municipal Corporation, and separately TSRTC in Telangana) already runs AI cameras on real public buses — 14 buses initially, scaling to 200 in Telangana — doing ADAS driver alerts, blackspot/greyspot mobility analysis, and road-infrastructure risk scoring. Judges familiar with Indian ITS work may already know this project, so don't pitch "bus-mounted AI camera for road safety" as if it's unprecedented — instead, explicitly differentiate:

- iRASTE is front-camera-only and driver-alert-focused (ADAS). Your pitch covers **multi-camera** (front/side/rear/cabin) with defect classification, vehicle density, ANPR/hit-and-run, and a **centralized command dashboard** with heatmaps and O-D analysis — a broader civic-infrastructure and law-enforcement scope than a driver-safety ADAS.
- You bring a working, field-validated IMU sensing layer they don't use, which becomes your bandwidth/triggering advantage.

6. Hardware/manufacturing angle for BEL specifically

BEL is a defense-electronics PSU (radar, optics, SATCOM, NavIC-adjacent systems). Positioning your onboard edge box as a ruggedized, BEL-manufacturable unit — potentially with NavIC integration alongside GPS — plays directly to BEL's own manufacturing and "Make in India" narrative, and is worth a slide even if you don't build the actual hardware for the hackathon.

7. Timeline reality check (as of Sept 11, 2026)

SIH 2026's national idea-submission deadline is around **30 September 2026** (uploaded by your SPOC), and that round is **screened purely by reading the 6-slide PPT — no live demo, nobody presents**. The Grand Finale (if shortlisted) is a **36-hour build in December**.

Practical implication:

- **For the PPT round:** prioritize a clear architecture diagram, the dataset table above (cite it — it signals you've done real homework), and screenshots/metrics from your *already-working* IMU app as proof of technical capability. This matters more right now than having every module coded.
 - **For the Grand Finale:** don't try to build all seven capabilities at production quality in 36 hours. Build a working demo of the Tier-1 items (pothole/defect fusion, vehicle density, ANPR + hit-and-run flagging, signboard detection) and show the rest (waterlogging, full O-D analytics, secure alerting) as an architected roadmap slide — this is normal and expected at SIH, not a weakness, as long as you're upfront about it.
-

8. Reference links

- RDD2020/2022 (CRDDC road damage): <https://github.com/sekilab/RoadDamageDetector>
- IDD / IDD-117K (IIIT-H): <https://idd.insaan.iiit.ac.in/>
- DriveIndia (TiHAN-IIT Hyderabad): <https://tihan.iith.ac.in/tiand-datasets/>
- IRTSD-Datasetv1 (Indian traffic signs): <https://ieee-dataport.org/documents/irtsd-datasetv1-indian-road-traffic-sign-detection-dataset>
- Indian Signboard/Traffic Sign datasets (DataCluster Labs): <https://github.com/datacluster-labs>
- Indian license plate dataset ("in the wild"): search "Indian Licence Plate Dataset in the wild" on academia.edu/ resulting GitHub repo
- AIS-140 / VLTD mandate background: https://en.wikipedia.org/wiki/Automotive_Industry_Standard_140
- iRASTE project: <https://www.iiit.ac.in/iraste/>